# **Specification of Python §4 Typed**—2026 edition

Martin Henz, Boyd Anderson

National University of Singapore
School of Computing

November 12, 2026

The language "Python §$x$" is a sublanguage of Python 3.13 and defined in the documents titled "Python §$x$", where $x$ refers to the respective textbook chapter.

## 1 Changes

Python §4 Typed is Python §4 with type annotations, which are checked before the program runs. Compared to Python §4, Python §4 Typed has the following changes:

- Function definitions annotate the types of their parameters and their return type, and assignments can declare the type of a name.
- Function definitions can have type parameters, and the new `type` statement defines type aliases.
- Programs are checked for type errors before they run, as described in Section "Static Type Checking" below. A program with a type error does not run.

A Python §4 Typed program without its annotations, type parameters and `type` statements is a Python §4 program, and it behaves in the same way.

## 2 Syntax

A Python program is a *program*, defined using Backus-Naur Form[1] as follows:

[1] We adopt Henry Ledgard's BNF variant that he described in *A human engineered variant of BNF*, ACM SIGPLAN Notices, Volume 15 Issue 10, October 1980, Pages 57-62. In our grammars, we use **`bold`** font for keywords, *italics* for syntactic variables, $\epsilon$ for nothing, $x \mid y$ for $x$ or $y$, [ $x$ ] for an optional $x$, $x$... for zero or more repetitions of $x$, and ( $x$ ) for clarifying the structure of BNF expressions.

| | | | |
|---|---|---|---|
| *program* | ::= | *import-stmt*... *block* | program |
| *import-stmt* | ::= | **from** *dotted-name* **import** *import-clause* | import statement |
| *dotted-name* | ::= | *name* ( **.** *name* )... | dotted identifier chain |
| *import-clause* | ::= | *import-as-names* | |
| | \| | **(** *import-as-names* **)** | wildcard or name list |
| *import-as-names* | ::= | *import-as-name* ( **,** *import-as-name* )... | clause name list |
| *import-as-name* | ::= | *name* [ **as** *name* ] | clause name and alias |
| *statement* | ::= | *name* [ **:** *type* ] **=** *expression* | assignment |
| | \| | **def** *name* [ *type-params* ] **(** *typed-names* **)** **->** *type* **:** *block* | function definition |
| | \| | **type** *name* [ *type-params* ] **=** *type* | type alias |
| | \| | **return** *expression* | return statement |
| | \| | *if-statement* | conditional statement |
| | \| | **while** *expression* **:** *block* | while loop |
| | \| | **for** *name* **in range(** *range-args* **) :** *block* | for loop |
| | \| | **break** | break statement |
| | \| | **continue** | continue statement |
| | \| | **global** *name* ( **,** *name* )... | global statement |
| | \| | **nonlocal** *name* ( **,** *name* )... | nonlocal statement |
| | \| | **pass** | pass statement |
| | \| | *expression* | expression statement |
| *range-args* | ::= | *expression* [ **,** *expression* ] [ **,** *expression* ] | for loop range arguments |
| *names* | ::= | $\epsilon$ \| *name* ( **,** *name* )... [ **,** *rest-name* ] | name list |
| *rest-name* | ::= | $\epsilon$ \| ***name** \| *name* ( **,** *name* )... [ **,** ***name** ] | name list (rest) |
| *if-statement* | ::= | **if** *expression* **:** *block* ( **elif** *expression* **:** *block* )... [ **else :** *block* ] | conditional statement |
| *block* | ::= | *statement*... | block statement |
| *expression* | ::= | *integer* | primitive integer expression |
| | \| | *float* | primitive float expression |
| | \| | *complex* | primitive complex expression |
| | \| | **True** \| **False** | primitive boolean expression |
| | \| | **None** | primitive None expression |
| | \| | *string* | primitive string expression |
| | \| | *name* | name expression |
| | \| | *expression binary-operator expression* | binary operator combination |
| | \| | *unary-operator expression* | unary operator combination |
| | \| | *expression binary-logical expression* | logical composition |
| | \| | *expression* **(** *spread-expressions* **)** | function application |
| | \| | **lambda** *names* **:** *expression* | lambda expression |
| | \| | *expression* **if** *expression* **else** *expression* | conditional expression |
| | \| | *expression***[***expression***] =** *expression* | list assignment |
| | \| | *expression***[***expression***]** | list access |
| | \| | **[** *expressions* **]** | literal list expression |
| | \| | **(** *expression* **)** | parenthesised expression |
| *binary-operator* | ::= | **+** \| **-** \| ***** \| **/** \| **//** \| ****** \| **%** \| **is** \| **is not** | |
| | \| | **==** \| **!=** \| **>** \| **<** \| **>=** \| **<=** | binary operator |
| *unary-operator* | ::= | **not** \| **-** \| **+** | unary operator |
| *binary-logical* | ::= | **and** \| **or** | logical composition symbol |
| *expressions* | ::= | $\epsilon$ \| *expression* ( **,** *expression* )... | element expressions |
| *spread-expressions* | ::= | $\epsilon$ \| *spread-expression* ( **,** *spread-expression* )... | argument expressions |
| *spread-expression* | ::= | *expression* \| ***** *expression* | argument expressions (spread) |

Parameters, type parameters and types are written as follows:

| | | | |
|---|---|---|---|
| *typed-names* | ::= | $\epsilon$ \| *typed-name* ( `,` *typed-name* )... [ `,` `*`*typed-name* ] | parameter list |
| | \| | `*`*typed-name* | |
| *typed-name* | ::= | *name* `:` *type* | annotated parameter |
| *type-params* | ::= | `[` *name* ( `,` *name* )... `]` | type parameters |
| *type* | ::= | *name* | named type |
| | \| | **`None`** | the type of `None` |
| | \| | *name* `[` *types* `]` | generic type |
| | \| | **`Callable[[`** [ *types* ] **`],`** *type* **`]`** | function type |
| | \| | *type* `\|` *type* | union type |
| *types* | ::= | *type* ( `,` *type* )... | type list |

## Indentation

In Python, indentation is syntactically significant and strictly enforces code block structure. Unlike languages using braces, Python employs whitespace (spaces or tabs) to delimit blocks for control flow (e.g., `if`, `for`, `while`), function definitions, and class declarations. Key rules:

- Consistency: Use 4 spaces per indentation level (PEP 8 recommendation). Mixing tabs and spaces is prohibited.
- Alignment: Statements within the same block must align vertically. Misaligned indentation raises an `IndentationError`.
- Nesting: Each nested block increases indentation by one level. Dedenting resumes the outer block.
- Line Continuation: For multi-line statements, align wrapped lines with the opening delimiter or use an extra level.

## Operator Precedence and Associativity

When an expression combines several operators, *precedence* decides which operator binds first, and *associativity* decides the order in which two operators of the same precedence group are applied. The table below lists the operators, function application, list access, the conditional expression (**`if` / `else`**), and **`lambda`** from the grammar above, in decreasing order of precedence: an entry binds more tighter than every entry below it. Use parentheses to override the default grouping.

| Operators | Description | Associativity |
|---|---|---|
| **`( ) [ ]`** | function application, list access | left to right |
| **`**`** | exponentiation | right to left |
| **`+ -`** | unary plus, unary minus | n/a |
| **`* / // %`** | multiplication, division, floor division, modulo | left to right |
| **`+ -`** | addition, subtraction | left to right |
| **`< > == != >= <=`** | comparison | n/a |
| **`is`**, **`is not`** | identity | n/a |
| **`not`** | logical negation | n/a |
| **`and`** | logical conjunction | left to right |
| **`or`** | logical disjunction | left to right |
| **`if` / `else`** | conditional expression | right to left |
| **`lambda`** | lambda expression | n/a |

Notes:

- Since **`**`** binds more tightly than unary **`+`**/**`-`**, the expression **`-2 ** 2`** is **`-(2 ** 2)`**, not **`(-2) ** 2`**.
- Comparison and identity operators are non-associative: unlike full Python, where e.g. **`a < b < c`** is equivalent to **`a < b and b < c`** (evaluating **`b`** once and short-circuiting before **`c`** is reached), this grammar gives every one of these operators exactly two operands. Chaining any of them, or mixing different ones as in **`a < b == c`**, is therefore rejected outright as a syntax error — a deliberate sublanguage restriction, rather than giving chained comparisons some other, non-standard meaning.
- Unary **`+`**/**`-`**, **`not`** and **`lambda`** take a single operand to their right, so stacking two of them has only one possible grouping (e.g. **`- -x`** always means **`-(-x)`**); associativity does not apply and is marked "n/a" for a different reason than for comparison operators above: here nothing is excluded, there is simply no choice to make.
- The conditional expression and **`lambda`** have the lowest precedence and extend as far to the right as possible; a **`lambda`** body or the **`else`** branch of a conditional expression may itself be another conditional expression or **`lambda`**.

## Restrictions

- All names that are referenced in a program need to be declared regardless of whether the reference gets evaluated or not.

  ```
  print(1 if True else x)
  ```

  Leads to an error `name not found in the current or enclosing environment(s).`

  A name is declared by an assignment, a `for` loop, a parameter or a function definition. The scope of a name that is declared in a function is that function, including the functions nested in it; a name that is declared at the top level is in scope everywhere in the program. A name that is declared in one function can therefore not be used in another function, unless the second function is nested in the first. A `global` declaration, anywhere in the program, declares the name at the top level (and not in the function that contains it), so that every function can use it, also a function that comes before the one with the declaration. Built-in names are in scope everywhere. The order of the declarations and the uses does not matter: a name that is used before it is assigned is not rejected, but leads to a run-time `NameError`, as in Python.

  ```
  def swap(y):
      print(x)     # x is declared global in g: swap can be defined first

  def g():
      global x
  ```

- In Python, a logical line is the smallest unit of code that the interpreter can execute, typically corresponding to a complete statement. A physical line, by contrast, is a single line in the source file, usually ending with a newline character. A logical line may span multiple physical lines depending on whether line joining is used.

  By default, each physical line corresponds to a logical line. However, when a statement is too long, Python allows it to be split across multiple physical lines using line continuation. Python provides two ways to join lines: **explicit line joining** and **implicit line joining**.

  **Explicit line joining** uses a backslash (\) to join the current physical line with the next one, forming a single logical line. The backslash must be the last character on the line—no characters (including whitespace or comments) are allowed after it.

  ```
  total = 1 + 2 + 3 + \
          4 + 5
  ```

  **Implicit line joining** occurs when expressions are enclosed in parentheses `()`, square brackets `[]`, or braces `{}`. Within these delimiters, line breaks are allowed without using a backslash.

  ```
  total = (
              1 + 2 + 3 +
              4 + 5
          )
  ```

  Therefore, when breaking a statement across lines, you must either use a backslash for explicit continuation or wrap the expression in parentheses, brackets, or braces for implicit continuation. Python requires each line to be syntactically complete—it does not infer the end of a statement implicitly.

  For details on logical lines, see the official Python documentation: Python Lexical Analysis – Logical Lines.

- Return statements are only allowed in bodies of functions. Each return statement must appear on a single logical line.

- Lambda expressions are limited to a single logical line.

## Loops

### `for` loop through a range with one argument

The expression in the following syntax should evaluate to an integer:

**`for`** *name* **`in range(`** *expression* **`) :`** *block*

This roughly stands for:

```
_loop_i = 0
_end = expression

while _loop_i < _end:
    name = _loop_i
    block
    _loop_i = _loop_i + 1
```

### `for` loop through a range with two arguments

The expressions in the following syntax should both evaluate to integers:

**`for`** *name* **`in range(`** $expression_1$ **`,`** $expression_2$ **`) :`** *block*

This roughly stands for:

```
_loop_i = expression_1
_end = expression_2

while _loop_i < _end:
    name = _loop_i
    block
    _loop_i = _loop_i + 1
```

### `for` loop through a range with three arguments

The expressions in the following syntax should all evaluate to integers:

**`for`** *name* **`in range(`** $expression_1$ **`,`** $expression_2$ **`,`** $expression_3$ **`) :`** *block*

This roughly stands for:

```
_loop_i = expression_1
_end = expression_2
_step = expression_3

if _step > 0:
    while _loop_i < _end:
        name = _loop_i
        block
        _loop_i = _loop_i + _step
else:
    while _loop_i > _end:
        name = _loop_i
        block
        _loop_i = _loop_i + _step
```

## Names

Names[2] start with _, or a letter[3] and contain only _, letters or digits[4]. Reserved words[5] are not allowed as names.
Valid names are `x`, `_45`, and `π`, but always keep in mind that programming is communicating and that the familiarity of the audience with the characters used in names is an important aspect of program readability.

## Numbers

Python supports three numeric types: integers (`int`), floats (`float`), and complex numbers (`complex`).
A numeric literal never includes a leading sign: a `+` or `-` written before a number, as in `-2` or `-3.14`, is unary-operator application (see "Operator Precedence and Associativity" above), not part of the literal itself. This is why **`-2 ** 2`** is **`-(2 ** 2)`** (i.e. $-4$) rather than **`(-2) ** 2`** (i.e. $4$): the sign is applied only after **`**`** has already bound the unsigned literal **`2`** to its exponent.

### Integers

Integers can be represented in decimal notation. Additional base notations are supported, such as binary (`0b1010` or `0B1010`), octal (`0o777` or `0O777`), and hexadecimal (`0x1A3F` or `0X1A3F`). Underscores (_) may be used for readability (e.g., `1_000_000`). Examples include `42`, `0b1101` (and, with a unary minus applied, `-0b1101`), and `0x_FF_00`.

### Floats

Floats use decimal notation with an optional decimal dot. Scientific notation (multiplying the number by $10^x$) is indicated with the letter `e` or `E`, followed by the exponent `x`. Special values inf (infinity), -inf, and nan (not a number) are allowed. Examples include `3.14`, `0.001e+05` (and, with a unary minus applied, `-0.001e+05`), and `6.022E23`.

### Complex Numbers

A complex value can be formed from a real part and an imaginary part, written `<real>±<imag>j`, where `j` (or `J`) denotes the imaginary unit. Both parts are stored as floats. The imaginary part is mandatory (e.g., `5j`, `0j`, and `0+3j` is valid, `5` alone is real). As with the sign of a real number, the `±` between the real and imaginary parts is not part of a single token: only `<imag>j` (e.g., `3j`) is a numeric literal (an *imaginary literal*), so `2+3j` parses as the real literal `2` combined with the imaginary literal `3j` by binary `+`, and `-4.5j` as unary `-` applied to the imaginary literal `4.5j`.
Examples include `2+3j`, `-4.5J`, `0j`, and `1e3-6.2E2J`.

## Strings

Strings are of the form `"`*double-quote-characters*`"`, where *double-quote-characters* is a possibly empty sequence of characters without the character `"` and without the newline character; of the form `'`*single-quote-characters*`'`, where *single-quote-characters* is a possibly empty sequence of characters without the character `'` and without the newline character; and of the form `'''`*triple-single-quote-characters*`'''`, or `"""`*triple-double-quote-characters*`"""`, where *triple-double-quote-characters* is a possibly empty sequence of characters and can span multiple lines and may contain both single and double quotes without escaping.
The following characters can be represented in strings[6] as given:

[2] In Python 3.13 Documentation, these names are called *identifiers*.
[3] By *letter* we mean Unicode letters (L) or letter numbers (Nl).
[4] By *digit* we mean characters in the Unicode categories Nd (including the decimal digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9), Mn, Mc and Pc.
[5] By *reserved word* we mean any of: **`False`**, **`await`**, **`else`**, **`import`**, **`pass`**, **`None`**, **`break`**, **`except`**, **`in`**, **`raise`**, **`True`**, **`class`**, **`finally`**, **`is`**, **`return`**, **`and`**, **`continue`**, **`for`**, **`lambda`**, **`try`**, **`as`**, **`def`**, **`from`**, **`nonlocal`**, **`while`**, **`assert`**, **`del`**, **`global`**, **`not`**, **`with`**, **`async`**, **`elif`**, **`if`**, **`or`**, **`yield`**. These are all reserved words, or keywords of the language that cannot be used as ordinary identifiers.
[6] In Python 3.13 Documentation, unless an 'r' or 'R' prefix is present, escape sequences in string and bytes literals are interpreted according to rules similar to those used by Standard C. *identifiers*.

- `\<newline>`: Backslash followed by a newline is ignored.
- `\\`: Represents a backslash (i.e., `\`).
- `\'`: Represents a single quote (').
- `\"`: Represents a double quote (").
- `\a`: Represents the ASCII Bell (BEL) character.
- `\b`: Represents the ASCII Backspace (BS) character.
- `\f`: Represents the ASCII Formfeed (FF) character.
- `\n`: Represents the ASCII Linefeed (LF) character.
- `\r`: Represents the ASCII Carriage Return (CR) character.
- `\t`: Represents the ASCII Horizontal Tab (TAB) character.
- `\v`: Represents the ASCII Vertical Tab (VT) character.

# 3 Static Type Checking

A Python §4 Typed program is checked for type errors before it runs. A program with a type error does not run; all its type errors are reported, each with the line and column where it occurs. Static type checking is in addition to the dynamic type checking described in the section "Dynamic Type Checking" below: a program that passes the static checks can still fail at run time, for example when it divides by zero or accesses a list outside its range.

## Types

The following types are available without imports.

| type | values |
|---|---|
| `int`, `float`, `complex` | numbers |
| `bool` | `True`, `False` |
| `str` | strings |
| `None` | `None` |
| `list[`$T$`]` | lists whose elements have type $T$ |
| `Pair[`$H$`, `$T$`]` | pairs whose head has type $H$ and whose tail has type $T$ |
| `LList[`$T$`]` | linked lists of elements of type $T$: `Pair[`$T$`, LList[`$T$`]] \| None` |
| `Stream[`$T$`]` | streams of elements of type $T$: `Pair[`$T$`, Callable[[], Stream[`$T$`]]] \| None` |
| `object` | any value: a function with a parameter of type `object`, such as `is_none`, accepts every value. Unlike `Any`, which is not available, an `object` can only be passed on or tested (`is_none(x)`, `is_pair(x)`, ...) |
| `Callable[[`$T_1$`, ..., `$T_n$`], `$R$`]` | functions with $n$ parameters of types $T_1, \ldots, T_n$ that return a value of type $R$ |
| $T_1$ `\|` $T_2$ | values of type $T_1$ or of type $T_2$ |

A pair is not a list as far as types are concerned, even though pairs are represented as lists of two elements: `pair(1, 2)[0]` is a type error, and the head and tail of a pair are accessed with `head` and `tail`. The elements of a list or a linked list all have the same type; a linked list with numbers and strings has a type such as `LList[int | str]`.
There is no type that stands for every type: the type `Any` of Python's `typing` module is not available. A function that works for values of any type has a type parameter (see below):

```
def identity[A](a: A) -> A:
    return a
```

Types need no import: `Callable`, `Pair`, `LList` and `Stream` are predeclared, and a program that imports from `typing` is rejected.

## Assignability

A value of type $S$ can be used where a value of type $T$ is expected if

- $S$ and $T$ are the same type, or $T$ is a union that includes $S$;
- $S$ is `int` and $T$ is `float` or `complex`, or $S$ is `float` and $T$ is `complex`;
- $S$ is `None` and $T$ is `LList[`$U$`]` or `Stream[`$U$`]` for any $U$;
- $S$ and $T$ are function types with the same number of parameters, each parameter type of $T$ can be used where the corresponding parameter type of $S$ is expected, and the result type of $S$ can be used where the result type of $T$ is expected.

In contrast to Python's usual type checkers, `bool` is not a number: `True` cannot be used where an `int` is expected. This follows the dynamic type checking of Python §4, where `True + 1` is an error. Lists and pairs can be changed, so their element types must match exactly: a `list[int]` cannot be used where a `list[float]` is expected, and a `Pair[int, int]` cannot be used where a `Pair[float, float]` is expected.

## Annotations

- Every function definition annotates each of its parameters and its return type. A function that does not return a value has the return type `None`.

```
def length(xs: LList[int]) -> int:
    return 0 if is_none(xs) else 1 + length(tail(xs))
```

- The parameters of a lambda expression are not annotated. Their types come from where the lambda expression is used: as an argument to a function whose parameter has a function type, or as the value of a name with a declared function type. A lambda expression whose parameter types cannot be determined that way is a type error.

```
ys = map(lambda x: x * 2, llist(1, 2, 3))      # x: int
square: Callable[[int], int] = lambda x: x * x
```

- An assignment can declare the type of a name: `count: int = 0`. Every value that is assigned to the name, also through `nonlocal` and `global`, must then have that type. A name without a declared type has, at each use, the type of the value that was assigned to it last; an empty list `[]` needs a declared type, such as `xs: list[int] = []`.

## Generic functions and type aliases

A function definition can introduce type parameters, which stand for any type. At each call, the type parameters are replaced by the types of the arguments.

```
def map_llist[A, B](f: Callable[[A], B], xs: LList[A]) -> LList[B]:
    return None if is_none(xs) else pair(f(head(xs)), map_llist(f, tail(xs)))
```

A type alias gives a name to a type. Type aliases can have type parameters and can be recursive.

```
type Rational = Pair[int, int]
type Tree[T] = Pair[T, LList[Tree[T]]]
```

## Conditions and narrowing

The condition after `if`, `elif` and `while`, the condition of a conditional expression, and the operand of `not` must have the type `bool`.
The left operand of `and` and `or` must have the type `bool` as well, but this is not checked statically: a program in which it has another type, such as `3 and 4`, has no static type error, and fails when it runs, as in Python §4 (see the section "Dynamic Type Checking" below).
In the branches of a conditional statement or conditional expression, the type of a name is narrowed according to the condition. If `xs` has the type `LList[`$T$`]`, it has the type `None` where `is_none(xs)` (or `xs is None`) is `True`, and the type `Pair[`$T$`, LList[`$T$`]]` where it is `False`, so

that `head(xs)` and `tail(xs)` are allowed there. The functions `is_pair`, `is_number`, `is_integer`, `is_float`, `is_complex`, `is_boolean`, `is_string`, `is_function` and `is_list` narrow types in the same way.

## Operators

The types of operator combinations follow the table in the section "Dynamic Type Checking" below, with these additions:

- $a$ `**` $b$ with `int` operands has the type `int` if $b$ is a non-negative integer literal, `float` if $b$ is a negative integer literal, and `int | float` otherwise; with a `float` operand and no `complex` operand, it has the type `float`.
- `==`, `!=`, `is` and `is not` accept operands of any types; `>`, `>=`, `<` and `<=` compare `bool` values with `int` and `float` values, as in Python §4.
- $a$ `and` $b$ and $a$ `or` $b$ have the type `bool | `$T$, where $T$ is the type of $b$.

The right operand of `and` and `or` is deliberately not required to have the type `bool`. Its value is the value of the whole expression, without any further check, so that a function whose result is $a$ `and` $f(\ldots)$ or $a$ `or` $f(\ldots)$ still gives rise to an iterative process, as in

```
def is_even(n: int) -> bool:
    return n == 0 or is_odd(n - 1)

def is_odd(n: int) -> bool:
    return n != 0 and is_even(n - 1)
```

Requiring the type `bool` for the right operand would add a check after the call of $f$, and the process would become recursive.

## Types of the predeclared functions

The predeclared functions have the following types, where $A$, $B$, $H$ and $T$ are type parameters.

```
pair(h: H, t: T) -> Pair[H, T]
head(p: Pair[H, T]) -> H,  tail(p: Pair[H, T]) -> T
set_head(p: Pair[H, T], v: H) -> None
set_tail(p: Pair[H, T], v: T) -> None
llist(*xs: A) -> LList[A],  length(xs: LList[A]) -> int
map(f: Callable[[A], B], xs: LList[A]) -> LList[B]
filter(pred: Callable[[A], bool], xs: LList[A]) -> LList[A]
reduce(f: Callable[[A, B], B], initial: B, xs: LList[A]) -> B
append(xs: LList[A], ys: LList[A]) -> LList[A]
reverse(xs: LList[A]) -> LList[A]
member(v: A, xs: LList[A]) -> LList[A]
llist_ref(xs: LList[A], n: int) -> A
enum_llist(start: int, end: int) -> LList[int]
stream_tail(s: Pair[H, Callable[[], T]]) -> T
stream_map(f: Callable[[A], B], s: Stream[A]) -> Stream[B]
is_none(x: object) -> bool   (narrows x; similarly the other is_ functions)
```

The types of the predeclared functions are listed in the documentation of Python §4 Typed, except those of `print`, `len`, `abs`, `max`, `min`, `round`, `input`, `repr`, `str` and `complex`, which have the types that Python's standard library gives them (with the difference that `bool` is not a number). The types of `tokenize`, `parse` and `apply_in_underlying_python`, and of the data that `parse` returns, are not yet fixed: they will be given as opaque syntax types with typed accessor functions (py-slang issue 631).

### Modules

A program can import names from the Source Academy modules whose types are known: `binary_tree`, `curve`, `matrix`, `repeat`, `rune`, `scrabble` and `sound`. The types are checked: a call that gives a function the wrong type of argument is an error, as for the functions of the program. A module whose types are not known cannot be imported: the import is an error.
The types that a module defines (`Rune` of `rune`, `Sound` of `sound`) are imported like the functions, to annotate with them: `from rune import Rune`. A name that is only a type is not part of the program that runs.
A number that a module's function takes can be an int or a float. A number that it returns is a float, and so is a number that it passes on to a function of the program.
A function of a module that works for any type (`repeat`, and the functions of `binary_tree`, which take the type of the entries of a tree as a type parameter) gives back the type that it was given: a tree with entries of type `int` gives an `int`. At run time an `int` that passes through such a function becomes a float, so a program that uses it where an `int` is required, as the index of a list, passes the check and fails when it runs (modules issue 998).

## 4 Dynamic Type Checking

Expressions evaluate to integers, floats, complex numbers, boolean values, strings or function values. Implementations of Source generated error messages when unexpected values are used as follows.
Only function values can be applied using the syntax:

*expression* ::= *name* **(** *expressions* **)**

For compound functions, implementations need to check that the number of *expressions* matches the number of parameters.
The following table specifies what arguments Python's operators take and what results they return. Implementations need to check the types of arguments and generate an error message when the types do not match.

| operator | argument 1 | argument 2 | result |
|---|---|---|---|
| `+,-,*,%,//` | int | int | int |
| `/` | int | int | float |
| `+,-,*,/,%,//` | int | float | float |
| `+,-,*,/,%,//` | float | int, float | float |
| `+,-,*,/` | int | complex | complex |
| `+,-,*,/` | float | complex | complex |
| `+,-,*,/` | complex | int, float, complex | complex |
| `+` | string | string | string |
| `**` | int | int >= 0 | int |
| `**` | int | int < 0 | float |
| `**` | int | float | float |
| `**` | int | complex | complex |
| `**` | float | int, float | float |
| `**` | float | complex | complex |
| `**` | complex | int, float, complex | complex |
| `*` | list | int | list |
| `*` | int | list | list |
| `==,!=,is,is not` | any | any | bool |
| `>,>=,<,<=` | int,float,bool | int,float,bool | bool |
| `>,>=,<,<=` | string | string | bool |
| `and` | bool | any | any |
| `or` | bool | any | any |
| `not` | bool | | bool |
| `-` | int | | int |
| `-` | float | | float |
| `-` | complex | | complex |

Following **`if`** and **`elif`**, Python §$x$ only allows boolean expressions. Lambda expressions can only be used as arguments to operators when they are parenthesized.

The operators **`==`**, **`!=`**, **`is`**, **`is not`**, **`>`**, **`>=`**, **`<`**, **`<=`** follow the Python 3.14 Documentation. As in Python, **`True`** and **`False`** are represented by the integers 1 and 0, respectively, for the purpose of any comparisons. The operators **`is`** and **`is not`** test for identity. As in Python, we can assume that there exists only a single instance of each of **`None`**, **`True`**, and **`False`** system-wide, and that all lists and functions have their own identity once created.

The operators **`==`** and **`!=`** on lists compare corresponding elements $x$ and $y$ as $x$ **`is`** $y$ **`or`** $x$ **`==`** $y$, and so does **`in`**, as in Python: an element is equal to itself, even if it is not equal to itself on its own. In particular, a **`nan`** that is referred to twice is equal to itself inside a list: `[math_nan] == [math_nan]` evaluates to **`True`**, whereas `math_nan == math_nan` evaluates to **`False`**. Whether two numbers that were computed separately are identical is left open, as in Python. In this language, numbers are identical if they have the same value, as in PyPy: a **`nan`** inside a list is equal to every **`nan`** inside the other list, also if the two were computed separately, which CPython does not consider equal.

List access *`list`*`[`*`index`*`]` and list assignment *`list`*`[`*`index`*`] = `*`value` require *index* to evaluate to an `int`; as opposed to Python, **`True`** and **`False`** are not allowed as indices. The valid range of *index* is `-len(`*list*`)` to `len(`*list*`) - 1`, inclusive, with negative indices wrapping around as in Python: if *list*`[`$i$`]` is defined, then *list*`[`$i$ `% len(`*list*`)]` gives the same value.

The operator **`*`** between a `list` and an `int` constructs a new list by repeating the elements of the given list the given number of times, with 0 or a negative number of times yielding **`[]`**; as opposed to Python, **`True`** and **`False`** are not allowed as the `int` operand. As in Python, the repetition makes shallow copies: `x = [[1,2]] * 4` makes `x[0] is x[1]` evaluate to **`True`**.

# 5 Standard Library

The standard library contains constants and functions that are always available in this language. In py-slang, the standard library functions are not implemented using a conventional split between primitive and predeclared functions. Unlike in Source, where predeclared functions are defined using built-in primitives, all standard library functions in py-slang are written directly in

TypeScript and embedded into the runtime of the CSE Machine. These functions are treated as part of the host environment rather than as user-level definitions, and they interact directly with the evaluator's internal control stack and environment. This design simplifies the interpreter architecture and allows for tighter integration with the execution model, enabling better support for visualization, error handling, and platform-specific extensions within the Source Academy learning environment.
The standard library for Python §$x$ is documented in online documentation of Python §$x$.
The types of the names that are described in the following sections are listed in the appendix "Types of the predeclared names"; the appendices on the linked list and stream libraries show their functions with their types.

## MISC Library

The following names are provided by the MISC library:

- `str(x)`: *primitive*, converts the value `x` to a complex number, or a string representation of `x` respectively, and returns the respective value.
- `repr(s)`: *primitive*, returns a string representation of `s` that is unambiguous and suitable for debugging; for many values, this string can be used to reconstruct `s`.
- `is_boolean(x)`, `is_integer(x)`, `is_float(x)`, `is_string(x)`, `is_none(x)`, `is_function(x)`, `is_complex(x)`: *primitive*, returns `True` if the type of `x` matches the function name and `False` if it does not.
- `complex(r, i)`: *primitive*, takes in two numbers `r` and `i`, and returns the complex number with real part `r` and imaginary part `i`.
- `real(x)`, `imag(x)`: *primitive*, return the real and imaginary parts of a complex number `x` respectively.
- `input(s)`: *primitive*, pops up a window that displays the *string* `s`, provides an input line for the user to enter a text, a "Cancel" button and an "OK" button. The call of `input` suspends execution of the program until one of the two buttons is pressed. If the "OK" button is pressed, `input` returns the entered text as a string. If the "Cancel" button is pressed, `input` returns a non-string value.
- `min(x, y, *args)` and `max(x, y, *args)`: *primitive*, takes in two or more arguments and returns the smallest and largest of the arguments respectively. If multiple arguments are equally small or large, the first one encountered in the argument list is returned. The arguments must be mutually comparable (i.e., they must support the < and > operators).
- `abs(x)`: *primitive*, returns the absolute value of `x`. For an `int` input, it returns the non-negative integer equivalent. For a `float` input, it returns the positive floating-point number representing its magnitude. For a `complex` input, it returns the modulus of the complex number, which is a non-negative float.
- `print(*x)`: *primitive*, takes in any number of parameters, converts them to their string representations using `str` and writes them to the console, followed by a newline character.
- `error(*x)`: *primitive*, takes in any number of parameters, converts them to their string representations using `str` and writes them to the console, followed by a newline character. The evaluation of any call of `error` aborts the running program immediately.
- `breakpoint()`: *primitive*, evaluates to the `None` value with no effect on the program. In the stepper and CSE Machine visualization, it additionally marks a breakpoint that the breakpoint navigation can jump to.
- `arity(f)`: *primitive*, takes in a function `f` and returns the minimum number of parameters that `f` can accept.
- `round(x)`[7]: *primitive*, takes in an int or float `x` and returns the nearest integer to `x`.

[7] In Python 3.13, when the value to be rounded is exactly halfway between two possible rounded values, it rounds to the nearest even number. For example, `round(2.5)` would return 2, while `round(3.5)` would return 4.

- `round(x, n)`: *primitive*, takes in an int or float `x` and an int `n`, and returns the value of `x` rounded to `n` digits of precision after the decimal point. If the second argument `n` is None, it is treated as `round(x)`.
- `len(s)`: *primitive*, takes in a container for a finite number of values (e.g., a string), and returns the number of elements in the container.
- `time_time()`: *primitive*, returns number of milliseconds elapsed since January 1, 1970 00:00:00 UTC
- `random_random()`: *primitive*, returns a random float uniformly in the range $[0.0, 1.0)$.

All library functions can be assumed to run in $O(1)$ time, except `print`, `error`, `str` and `repr`, which run in $O(n)$ time, where $n$ is the size (number of components such as pairs) of their arguments.

### MATH Library

The following names are provided by the MATH library:

- `math_`*name*, where *name* is any name specified in the Python **`math`** library, see Python 3.14 Documentation. Examples:
  - `math_nan`: *primitive*, refers to the NaN ("Not a Number") value,
  - `math_inf`: *primitive*, refers to the Infinity value,
  - `math_pi`: *primitive*, refers to the mathematical constant $\pi$,
  - `math_sqrt(n)`: *primitive*, returns the square root of the *number* `n`.

All functions can be assumed to run in $O(1)$ time and are considered *primitive*.

## Appendix: Linked List library

Those linked list library functions that are not primitive functions are pre-declared as follows, with their types. (The type of a function that stands for any type has type parameters; `X1` stands for a type that does not matter there. The functions whose names start with an underscore are helper functions.)

```
def pair[H, T](x: H, y: T) -> Pair[H, T]:
    """
    PRIMITIVE
    Makes a pair whose head (first component) is x and whose
    tail (second component) is y.

    Parameters:
        x (value): given head
        y (value): given tail

    Returns:
        pair: pair with x as head and y as tail
    """
    pass


def is_pair(x: object) -> bool:
    """
    PRIMITIVE
    Returns True if x is a pair and False otherwise.

    Parameters:
        x (value): given value
```

```
    Returns:
        boolean: whether x is a pair
    """
    pass


def head[H, T](p: Pair[H, T]) -> H:
    """
    PRIMITIVE
    Returns head (first component) of given pair p.

    Parameters:
        p (pair): given pair

    Returns:
        value: head of p
    """
    pass


def tail[H, T](p: Pair[H, T]) -> T:
    """
    PRIMITIVE
    Returns tail (second component) of given pair p.

    Parameters:
        p (pair): given pair

    Returns:
        value: tail of p
    """
    pass


def is_none(x: object) -> bool:
    """
    PRIMITIVE
    Returns True if x is the empty linked list None, and False otherwise.

    Parameters:
        x (value): given value

    Returns:
        boolean: whether x is None
    """
    pass

def llist[A](*values: A) -> LList[A]:
    """
    PRIMITIVE
    Given n values, returns a linked list of length n. The elements of
    the linked list are the given values in the given order.

    Parameters:
        value (value1, value2, ...values): given values

    Returns:
        linked list: linked list containing all values
    """
    pass
```

```
def draw_data[A](value1: A, *values: object) -> A:
    """
    PRIMITIVE
    Visualizes the arguments in a separate drawing area in the Source
    Academy using box-and-pointer diagrams.

    Parameters:
        value1 (value): first value to visualize
        values (value1, value2, ...): additional values to visualize
    """
    pass

def is_llist(xs: object) -> bool:
    """
    Returns True if xs is a linked list as defined in the textbook, and
    False otherwise.

    Parameters:
        xs (value): given value

    Returns:
        boolean: whether xs is a linked list
    """
    if is_none(xs):
        return True
    elif is_pair(xs):
        return is_llist(tail(xs))
    else:
        return False

def length[A](xs: LList[A]) -> int:
    """
    Returns the length of the linked list xs.
    """
    return _length(xs, 0)

def _length[A](xs: LList[A], acc: int) -> int:
    return acc if is_none(xs) else _length(tail(xs), acc + 1)

def map[A, B](f: Callable[[A], B], xs: LList[A]) -> LList[B]:
    """
    Returns a linked list that results from linked list xs by element-wise
    application of unary function f.
    """
    return _map(f, xs, None)

def _map[A, B](f: Callable[[A], B], xs: LList[A], acc: LList[B]) -> LList[B]:
    return (
        reverse(acc)
        if is_none(xs)
        else _map(f, tail(xs), pair(f(head(xs)), acc))
    )

def build_llist[A](fun: Callable[[int], A], n: int) -> LList[A]:
    """
    Makes a linked list with n elements by applying the unary function fun
    to the numbers 0 to n - 1.
    """
    return _build_llist(n - 1, fun, None)
```

```
def _build_llist[A](i: int, fun: Callable[[int], A], already_built: LList[A]) -> LList[A
    return (
        already_built
        if i < 0
        else _build_llist(i - 1, fun, pair(fun(i), already_built))
    )

def for_each[A](fun: Callable[[A], object], xs: LList[A]) -> bool:
    """
    Applies the unary function fun to every element of the linked list xs.
    """
    if is_none(xs):
        return True
    else:
        fun(head(xs))  # Side effect happens here if fun is not pure
        return for_each(fun, tail(xs))

def llist_to_string[A](xs: LList[A]) -> str:
    """
    Returns a string that represents linked list xs using the text-based
    box-and-pointer notation.
    """
    return _llist_to_string(xs, lambda x: x)

def _llist_to_string(xs: object, cont: Callable[[str], str]) -> str:
    if is_none(xs):
        return cont("None")
    elif is_pair(xs):
        return _llist_to_string(
            head(xs),
            lambda x_str: _llist_to_string(
                tail(xs), lambda y_str: cont("[" + x_str + ", " + y_str + "]")
            ),
        )
    else:
        return cont(repr(xs))

def reverse[A](xs: LList[A]) -> LList[A]:
    """
    Returns linked list xs in reverse order.
    """
    return _reverse(xs, None)

def _reverse[A](original: LList[A], reversed_acc: LList[A]) -> LList[A]:
    return (
        reversed_acc
        if is_none(original)
        else _reverse(
            tail(original), pair(head(original), reversed_acc)
        )
    )

def append[A](xs: LList[A], ys: LList[A]) -> LList[A]:
    """
    Returns a linked list that results from appending the linked list ys
    to the linked list xs.
    """
    return _append(xs, ys, lambda x: x)
```

```
def _append[A](xs: LList[A], ys: LList[A], cont: Callable[[LList[A]], LList[A]]) -> LLis
    return (
        cont(ys)
        if is_none(xs)
        else _append(
            tail(xs), ys, lambda zs: cont(pair(head(xs), zs))
        )
    )

def member[A](v: A, xs: LList[A]) -> LList[A]:
    """
    Returns first postfix sub-linked list whose head is identical to v
    (using ==). Returns None if the element does not occur in the linked
    list.
    """
    if is_none(xs):
        return None
    elif v == head(xs):
        return xs
    else:
        return member(v, tail(xs))

def remove[A](v: A, xs: LList[A]) -> LList[A]:
    """
    Returns a linked list that results from xs by removing the first item
    from xs that is identical (==) to v.
    """
    return _remove(v, xs, None)

def _remove[A](v: A, xs: LList[A], acc: LList[A]) -> LList[A]:
    if is_none(xs):
        return append(reverse(acc), xs)
    elif v == head(xs):
        return append(reverse(acc), tail(xs))
    else:
        return _remove(v, tail(xs), pair(head(xs), acc))

def remove_all[A](v: A, xs: LList[A]) -> LList[A]:
    """
    Returns a linked list that results from xs by removing all items from
    xs that are identical (==) to v.
    """
    return _remove_all(v, xs, None)

def _remove_all[A](v: A, xs: LList[A], acc: LList[A]) -> LList[A]:
    if is_none(xs):
        return append(reverse(acc), xs)
    elif v == head(xs):
        return _remove_all(v, tail(xs), acc)
    else:
        return _remove_all(v, tail(xs), pair(head(xs), acc))

def enum_llist(start: int, end: int) -> LList[int]:
    """
    Makes a linked list with elements from start to end (inclusive).
    """
    return _enum_llist(start, end, None)

def _enum_llist(start: int, end: int, acc: LList[int]) -> LList[int]:
    return (
```

```
        reverse(acc)
        if start > end
        else _enum_llist(start + 1, end, pair(start, acc))
    )

def llist_ref[A](xs: LList[A], n: int) -> A:
    """
    Returns the element of linked list xs at position n (0-indexed).
    """
    if n == 0:
        if is_none(xs):
            error("llist_ref: index out of bounds on None linked list")
        return head(xs)
    else:
        if is_none(xs):
            error("llist_ref: index out of bounds")
        return llist_ref(tail(xs), n - 1)


def reduce[A, B](f: Callable[[A, B], B], initial: B, xs: LList[A]) -> B:
    """
    Applies binary function f to the elements of xs from right-to-left
    order.
    """
    return _reduce(f, initial, xs, lambda x: x)


def _reduce[A, B](f: Callable[[A, B], B], initial: B, xs: LList[A], cont: Callable[[B],
    if is_none(xs):
        return cont(initial)
    else:
        # Recursive CPS call: Process tail, then apply f with head,
        # then pass to continuation
        return _reduce(
            f,
            initial,
            tail(xs),
            lambda x_reduced_from_tail: cont(
                f(head(xs), x_reduced_from_tail)
            ),
        )


def filter[A](pred: Callable[[A], bool], xs: LList[A]) -> LList[A]:
    """
    Returns a linked list that contains only those elements for which the
    one-argument function pred returns True.
    """
    return _filter(pred, xs, None)


def _filter[A](pred: Callable[[A], bool], xs: LList[A], acc: LList[A]) -> LList[A]:
    if is_none(xs):
        return reverse(acc)
    else:
        if pred(head(xs)):
            return _filter(pred, tail(xs), pair(head(xs), acc))
        else:
            return _filter(pred, tail(xs), acc)
```

### Pair Mutators

The following pair mutator functions are supported:

- `set_head(p, x)`: *primitive*, changes the pair `p` such that its head is `x`. Returns `None`.
- `set_tail(p, x)`: *primitive*, changes the pair `p` such that its tail is `x`. Returns `None`.

## Appendix: Stream library

Those stream library functions that are not primitive functions are pre-declared as follows, with their types. The functions `stream_ref` and `eval_stream` assume that the stream has enough elements: a program of Python §4 Typed that defines them in this way has a type error, because the stream can be `None`.

```
def stream_tail[H, T](xs: Pair[H, Callable[[], T]]) -> T:
    """
    stream_tail returns the second component of the given pair
    throws an error if the argument is not a pair
    """
    if is_pair(xs):
        the_tail = tail(xs)
        if is_function(the_tail):
            return the_tail()
        else:
            error(
                the_tail,
                "stream_tail(xs) expects a function as "
                + "the tail of the argument pair xs, "
                + "but encountered ",
            )

    else:
        error(
            xs,
            "stream_tail(xs) expects a pair as "
            + "argument xs, but encountered ",
        )


def is_stream(xs: object) -> bool:
    """
    is_stream recurses down the stream and checks that it ends with the
    empty linked list None
    """
    return is_none(xs) or (
        is_pair(xs)
        and is_function(tail(xs))
        and arity(tail(xs)) == 0
        and is_stream(stream_tail(xs))
    )


def llist_to_stream[A](xs: LList[A]) -> Stream[A]:
    """
    A stream is either None or a pair whose tail is
    a nullary function that returns a stream.
    """
    return (
        None
        if is_none(xs)
```

```
        else pair(head(xs), lambda: llist_to_stream(tail(xs)))
    )


def stream_to_llist[A](xs: Stream[A]) -> LList[A]:
    """
    stream_to_llist transforms a given stream to a linked list
    Lazy? No: stream_to_llist needs to force the whole stream
    """
    return (
        None if is_none(xs) else pair(head(xs), stream_to_llist(stream_tail(xs)))
    )


def stream_length[A](xs: Stream[A]) -> int:
    """
    stream_length returns the length of a given argument stream
    throws an exception if the argument is not a stream
    Lazy? No: The function needs to explore the whole stream
    """
    return 0 if is_none(xs) else 1 + stream_length(stream_tail(xs))


def stream_map[A, B](f: Callable[[A], B], s: Stream[A]) -> Stream[B]:
    """
    stream_map applies first arg f to the elements of the second
    argument, assumed to be a stream.
    f is applied element-by-element:
    stream_map(f, llist_to_stream(llist(1,2)) results in
    the same as llist_to_stream(llist(f(1),f(2)))
    stream_map throws an exception if the second argument is not a
    stream, and if the second argument is a nonempty stream and the
    first argument is not a function.
    Lazy? Yes: The argument stream is only explored as forced by
                        the result stream.
    """
    return (
        None
        if is_none(s)
        else pair(f(head(s)), lambda: stream_map(f, stream_tail(s)))
    )


def build_stream[A](fun: Callable[[int], A], n: int) -> Stream[A]:
    """
    build_stream takes a function fun as first argument,
    and a nonnegative integer n as second argument,
    build_stream returns a stream of n elements, that results from
    applying fun to the numbers from 0 to n-1.
    Lazy? Yes: The result stream forces the applications of fun
                        for the next element
    """

    def build(i: int) -> Stream[A]:
        return None if i >= n else pair(fun(i), lambda: build(i + 1))

    return build(0)


def stream_for_each[A](fun: Callable[[A], object], xs: Stream[A]) -> bool:
```

```
    """
    stream_for_each applies first arg fun to the elements of the stream
    passed as second argument. fun is applied element-by-element:
    for_each(fun, llist_to_stream(llist(1, 2, None))) results in the calls fun(1)
    and fun(2).
    stream_for_each returns True.
    stream_for_each throws an exception if the second argument is not a
    stream, and if the second argument is a nonempty stream and the
    first argument is not a function.
    Lazy? No: stream_for_each forces the exploration of the entire stream
    """
    if is_none(xs):
        return True
    else:
        fun(head(xs))
        return stream_for_each(fun, stream_tail(xs))


def stream_reverse[A](xs: Stream[A]) -> Stream[A]:
    """
    stream_reverse reverses the argument stream
    stream_reverse throws an exception if the argument is not a stream.
    Lazy? No: stream_reverse forces the exploration of the entire stream
    """

    def rev(original: Stream[A], reversed: Stream[A]) -> Stream[A]:
        return (
            reversed
            if is_none(original)
            else rev(
                stream_tail(original), pair(head(original), lambda: reversed)
            )
        )

    return rev(xs, None)


def stream_append[A](xs: Stream[A], ys: Stream[A]) -> Stream[A]:
    """
    stream_append appends first argument stream and second argument stream.
    In the result, None at the end of the first argument stream
    is replaced by the second argument stream
    stream_append throws an exception if the first argument is not a
    stream.
    Lazy? Yes: the result stream forces the actual append operation
    """
    return (
        ys
        if is_none(xs)
        else pair(head(xs), lambda: stream_append(stream_tail(xs), ys))
    )


def stream_member[A](x: A, s: Stream[A]) -> Stream[A]:
    """
    stream_member looks for a given first-argument element in a given
    second argument stream. It returns the first postfix substream
    that starts with the given element. It returns None if the
    element does not occur in the stream
    Lazy? Sort-of: stream_member forces the stream only until the element is found.
```

```
    """
    return (
        None
        if is_none(s)
        else s
        if head(s) == x
        else stream_member(x, stream_tail(s))
    )


def stream_remove[A](v: A, xs: Stream[A]) -> Stream[A]:
    """
    stream_remove removes the first occurrence of a given first-argument element
    in a given second-argument linked list. Returns the original linked list
    if there is no occurrence.
    Lazy? Yes: the result stream forces the construction of each next element
    """
    return (
        None
        if is_none(xs)
        else stream_tail(xs)
        if v == head(xs)
        else pair(head(xs), lambda: stream_remove(v, stream_tail(xs)))
    )


def stream_remove_all[A](v: A, xs: Stream[A]) -> Stream[A]:
    """
    stream_remove_all removes all instances of v instead of just the first.
    Lazy? Yes: the result stream forces the construction of each next element
    """
    return (
        None
        if is_none(xs)
        else stream_remove_all(v, stream_tail(xs))
        if v == head(xs)
        else pair(head(xs), lambda: stream_remove_all(v, stream_tail(xs)))
    )


def stream_filter[A](p: Callable[[A], bool], s: Stream[A]) -> Stream[A]:
    """
    filter returns the substream of elements of given stream s
    for which the given predicate function p returns True.
    Lazy? Yes: The result stream forces the construction of
    each next element. Of course, the construction
    of the next element needs to go down the stream
    until an element is found for which p holds.
    """
    return (
        None
        if is_none(s)
        else pair(head(s), lambda: stream_filter(p, stream_tail(s)))
        if p(head(s))
        else stream_filter(p, stream_tail(s))
    )


def enum_stream(start: int, end: int) -> Stream[int]:
    """
```

```
    enumerates numbers starting from start,
    using a step size of 1, until the number
    exceeds end.
    Lazy? Yes: The result stream forces the construction of
    each next element
    """
    return (
        None
        if start > end
        else pair(start, lambda: enum_stream(start + 1, end))
    )


def integers_from(n: int) -> Stream[int]:
    """
    integers_from constructs an infinite stream of integers
    starting at a given number n
    Lazy? Yes: The result stream forces the construction of
    each next element
    """
    return pair(n, lambda: integers_from(n + 1))


def eval_stream[A](s: Stream[A], n: int) -> LList[A]:
    """
    eval_stream constructs the linked list of the first n elements
    of a given stream s
    Lazy? Sort-of: eval_stream only forces the computation of
    the first n elements, and leaves the rest of
    the stream untouched.
    """

    def es(s: Stream[A], n: int) -> LList[A]:
        return (
            llist(head(s))
            if n == 1
            else pair(head(s), es(stream_tail(s), n - 1))
        )

    return None if n == 0 else es(s, n)


def stream_ref[A](s: Stream[A], n: int) -> A:
    """
    Returns the item in stream s at index n (the first item is at position 0)
    Lazy? Sort-of: stream_ref only forces the computation of
    the first n elements, and leaves the rest of
    the stream untouched.
    """
    return head(s) if n == 0 else stream_ref(stream_tail(s), n - 1)
```

## Interpreter Support

- `apply_in_underlying_python(f, xs)`: *primitive*, calls the function `f` with arguments `xs`. For example:

```
def times(x, y):
    return x * y
apply_in_underlying_python(times, llist(2, 3))  # returns 6
```

- `tokenize(x)`: *primitive*, returns the literal linked list of tokens that results from tokenizing the string `x` as a Python program. Each token is a string that contains the characters of the token as they appear in the program. Comments are ignored.
- `parse(x)`: *primitive*, returns the parse tree that results from parsing the string `x` as a Python program. The following two pages describe the shape of the parse tree. The tree is represented by the tagged linked lists on the right; the angle brackets denote recursive application of the transformation rules. Implementations are allowed to support more of Python than listed.

In addition, the Source Academy frontend predeclares the name `__program__` in all Python languages to refer to the string representation of the entrypoint file of the program in the editor that is being run using "Run". The entrypoint file is the file containing the code that acts as the entrypoint of the program being run. If `__program__` is used in the REPL, it refers to the string representation of the editor content at the time when "Run" was last pressed.

| | | |
|---|---|---|
| *program*::= | *statement* ... | `llist("sequence", llist of` ⟨*statement*⟩`)`; a single statement stands for itself |
| *statement*::= | *name* **=** *expression* | `llist("assignment",` ⟨*name*⟩, ⟨*expression*⟩`)` |
| \| | **def** *name* **(** *names* **)** **:** *body* | `llist("function_definition",` ⟨*name*⟩, ⟨*names*⟩, ⟨*body*⟩`)` |
| \| | **return** *expression* | `llist("return_statement",` ⟨*expression*⟩`)` |
| \| | *if-statement* | see below |
| \| | **while (** *expression* **)** **:** *body* | `llist("while_loop",` ⟨*expression*⟩, ⟨*body*⟩`)` |
| \| | **for** *name* **in range** **(** *range-args* **)** **:** *body* | `llist("for_loop",` ⟨*name*⟩, ⟨*range_args*⟩, ⟨*body*⟩`)` |
| \| | **break** | `llist("break_statement")` |
| \| | **continue** | `llist("continue_statement")` |
| \| | **global** *names* | `llist("global_statement",` ⟨*names*⟩`)` |
| \| | **nonlocal** *names* | `llist("nonlocal_statement",` ⟨*names*⟩`)` |
| \| | **pass** | `llist("pass_statement")` |
| \| | *expression* | see below |
| *range_args*::= | $expression_1$ **[**, $expression_2$ **]** **[** , $expression_3$ **]** | `llist("range_args",` ⟨$expression_1$⟩, ⟨$expression_2$⟩, ⟨$expression_3$⟩`)` |
| *names*::= | $\epsilon$ \| *name* ( **,** *name* ) ... | `llist of` ⟨*name*⟩ |
| *if-statement*::= | **if** $expression_1$ **:** $body_1$ ( **elif** $expression_2$ **:** $body_2$ )... **else :** $body_3$ | `llist("conditional_statement",` ⟨$expression_1$⟩, ⟨$body_1$⟩, ⟨$body_3$⟩ or ⟨*if-statement*⟩`)`; a missing else part is `llist("pass_statement")` |
| *body*::= | *statement*... | `llist("sequence", llist of` ⟨*statement*⟩`)`; a single statement stands for itself |

| | | |
|---|---|---|
| *expression*.::= | *integer* | `llist("literal", `*integer*`)` |
| \| | *float* | `llist("literal", `*float*`)` |
| \| | *complex* | `llist("literal", `*complex*`)` |
| \| | **`True`** \| **`False`** | `llist("literal", `*True*`)` |
| \| | **`None`** | `llist("literal", `*None*`)` |
| \| | *string* | `llist("literal", `*string*`)` |
| \| | *name* | `llist("name", string)` |
| \| | *expression*$_1$ *log-op expression*$_2$ | `llist("logical_composition",` ⟨*log-op*⟩, ⟨*expression*$_1$⟩, ⟨*expression*$_2$⟩`)` |
| \| | *expression*$_1$ *bin-op expression*$_2$ | `llist("binary_operator_combination",` ⟨*bin-op*⟩, ⟨*expression*$_1$⟩, ⟨*expression*$_2$⟩`)` |
| \| | *un-op expression* | `llist("unary_operator_combination",` ⟨*un-op*⟩, ⟨*expression*⟩`)` |
| \| | *expression* **`(`** *expressions* **`)`** | `llist("application",` ⟨*expression*⟩, *llist of* ⟨*expression*⟩`)` |
| \| | **`lambda`** *names* **`:`** *expression* | `llist("lambda_expression",` ⟨*names*⟩, ⟨*expression*⟩`)` |
| \| | *expression*$_1$ **`if`** *expression*$_2$ **`else`** *expression*$_3$ | `llist("conditional_expression",` ⟨*expression*$_1$⟩, ⟨*expression*$_2$⟩, ⟨*expression*$_3$⟩`)` |
| \| | *expression*$_1$ **`[`***expression*$_2$**`]=`** *expression*$_3$ | `llist("list_assignment",` ⟨*expression*$_1$⟩, ⟨*expression*$_2$⟩, ⟨*expression*$_3$⟩`)` |
| \| | *expression*$_1$ **`[`***expression*$_2$**`]`** | `llist("list_access",` ⟨*expression*$_1$⟩, ⟨*expression*$_2$⟩`)` |
| \| | **`[`** *expressions* **`]`** | `llist("list_expression", llist of` ⟨*expression*⟩`)` |
| \| | **`(`** *expression* **`)`** | parenthesised *expression* |
| *log-op*.::= | **`and`** \| **`or`** | *string representing operator* |
| *bin-op*::= | **`+`** \| **`-`** \| **`*`** \| **`/`** \| **`//`** \| **`**`** \| **`%`** \| **`is`** \| **`is not`** | *string representing operator* |
| \| | **`==`** \| **`!=`** \| **`>`** \| **`<`** \| **`>=`** \| **`<=`** | *string representing operator* |
| *un-op*::= | **`not`** \| **`-`** | `"-unary"` |
| \| | **`+`** | `"+unary"` |
| *expressions*::= | $\epsilon$ \| *expression* ( **`,`** *expression* ) ... | `llist of` ⟨*expression*⟩ |

# Appendix: Types of the predeclared names

The predeclared names have the following types. `TypeIs[T]` means `bool`, and in addition narrows the type of the argument to `T`; `TypeGuard[T]` also means `bool`, but narrows the type only where the result is true. `is_float` and `is_complex` have it, because the checker takes an `int` for a `float` and a `complex`, although `is_float(5)` is false. `NoReturn` is the result of a function that never returns. In the `TypeIs` of `is_pair` and `is_list`, `object` stands for the types of the components and elements, which the checker does not know: it accepts any type there, so that narrowing a union keeps its pairs and its lists. `tail` and `stream_tail` have two declarations: the first is for linked lists and streams, so that the tail of an `LList[int]` is an `LList[int]`, and the second is for any other pair. The names `print`, `len`, `abs`, `max`, `min`, `round`, `input`, `repr`, `str` and `complex` have the types that Python's standard library gives them (but `bool` is not a number).

## misc

```
def error(*object: object) -> NoReturn
def breakpoint() -> None
def time_time() -> float
def random_random() -> float
def arity(f: Callable[..., object]) -> int
def real(x: complex) -> float
def imag(x: complex) -> float
def set_timeout(f: Callable[[], object], t: int | float) -> None
def clear_all_timeout() -> None
def is_none(x: object) -> TypeIs[None]
def is_boolean(x: object) -> TypeIs[bool]
def is_integer(x: object) -> TypeIs[int]
def is_float(x: object) -> TypeGuard[float]
def is_complex(x: object) -> TypeGuard[complex]
def is_number(x: object) -> TypeIs[int | float | complex]
def is_string(x: object) -> TypeIs[str]
def is_function(x: object) -> TypeIs[Callable[..., object]]
def is_list(x: object) -> TypeIs[list[object]]
```

## pairs and linked lists

```
def pair[H, T](h: H, t: T) -> Pair[H, T]
def head[H, T](p: Pair[H, T]) -> H
def tail[A](p: Pair[A, LList[A]]) -> LList[A]
def tail[H, T](p: Pair[H, T]) -> T
```

```
def is_pair(x: object) -> TypeIs[Pair[object, object]]
def llist[A](*xs: A) -> LList[A]
def is_llist(x: object) -> bool
def print_llist[A](xs: LList[A]) -> None
def llist_to_string[A](xs: LList[A]) -> str
def length[A](xs: LList[A]) -> int
def map[A, B](f: Callable[[A], B], xs: LList[A]) -> LList[B]
def filter[A](pred: Callable[[A], bool], xs: LList[A]) -> LList[A]
def reduce[A, B](f: Callable[[A, B], B], initial: B, xs: LList[A]) -> B
def for_each[A](f: Callable[[A], object], xs: LList[A]) -> bool
def build_llist[A](f: Callable[[int], A], n: int) -> LList[A]
def enum_llist(start: int, end: int) -> LList[int]
def reverse[A](xs: LList[A]) -> LList[A]
def append[A](xs: LList[A], ys: LList[A]) -> LList[A]
def member[A](v: A, xs: LList[A]) -> LList[A]
def remove[A](v: A, xs: LList[A]) -> LList[A]
def remove_all[A](v: A, xs: LList[A]) -> LList[A]
def llist_ref[A](xs: LList[A], n: int) -> A
def draw_data[A](value1: A, *values: object) -> A
```

### pair mutators

```
def set_head[H, T](p: Pair[H, T], v: H) -> None
def set_tail[H, T](p: Pair[H, T], v: T) -> None
```

### streams

```
def stream[A](*xs: A) -> Stream[A]
def is_stream(x: object) -> bool
def stream_tail[A](s: Pair[A, Callable[[], Stream[A]]]) -> Stream[A]
def stream_tail[H, T](s: Pair[H, Callable[[], T]]) -> T
def stream_ref[A](s: Stream[A], n: int) -> A
def stream_length[A](s: Stream[A]) -> int
def stream_map[A, B](f: Callable[[A], B], s: Stream[A]) -> Stream[B]
def stream_filter[A](pred: Callable[[A], bool], s: Stream[A]) -> Stream[A]
def stream_for_each[A](f: Callable[[A], object], s: Stream[A]) -> bool
def stream_reverse[A](s: Stream[A]) -> Stream[A]
def stream_append[A](s: Stream[A], t: Stream[A]) -> Stream[A]
def stream_member[A](v: A, s: Stream[A]) -> Stream[A]
def stream_remove[A](v: A, s: Stream[A]) -> Stream[A]
def stream_remove_all[A](v: A, s: Stream[A]) -> Stream[A]
def build_stream[A](f: Callable[[int], A], n: int) -> Stream[A]
def enum_stream(start: int, end: int) -> Stream[int]
def integers_from(start: int) -> Stream[int]
def llist_to_stream[A](xs: LList[A]) -> Stream[A]
def stream_to_llist[A](s: Stream[A]) -> LList[A]
def eval_stream[A](s: Stream[A], n: int) -> LList[A]
```

### math

```
math_e: float
math_inf: float
math_nan: float
math_pi: float
math_tau: float
def math_acos(x: int | float) -> float
def math_acosh(x: int | float) -> float
```

```
def math_asin(x: int | float) -> float
def math_asinh(x: int | float) -> float
def math_atan(x: int | float) -> float
def math_atan2(y: int | float, x: int | float) -> float
def math_atanh(x: int | float) -> float
def math_cbrt(x: int | float) -> float
def math_ceil(x: int | float) -> int
def math_comb(n: int, k: int) -> int
def math_copysign(x: int | float, y: int | float) -> float
def math_cos(x: int | float) -> float
def math_cosh(x: int | float) -> float
def math_degrees(x: int | float) -> float
def math_erf(x: int | float) -> float
def math_erfc(x: int | float) -> float
def math_exp(x: int | float) -> float
def math_exp2(x: int | float) -> float
def math_expm1(x: int | float) -> float
def math_fabs(x: int | float) -> float
def math_factorial(n: int) -> int
def math_floor(x: int | float) -> int
def math_fma(x: int | float, y: int | float, z: int | float) -> float
def math_fmod(x: int | float, y: int | float) -> float
def math_gamma(x: int | float) -> float
def math_gcd(*integers: int) -> int
def math_isfinite(x: int | float) -> bool
def math_isinf(x: int | float) -> bool
def math_isnan(x: int | float) -> bool
def math_isqrt(n: int) -> int
def math_lcm(*integers: int) -> int
def math_ldexp(x: int | float, i: int) -> float
def math_lgamma(x: int | float) -> float
def math_log(x: int | float, base: int | float = ...) -> float
def math_log10(x: int | float) -> float
def math_log1p(x: int | float) -> float
def math_log2(x: int | float) -> float
def math_nextafter(x: int | float, y: int | float, steps: int = ...) -> float
def math_perm(n: int, k: int = ...) -> int
def math_pow(x: int | float, y: int | float) -> float
def math_radians(x: int | float) -> float
def math_remainder(x: int | float, y: int | float) -> float
def math_sin(x: int | float) -> float
def math_sinh(x: int | float) -> float
def math_sqrt(x: int | float) -> float
def math_tan(x: int | float) -> float
def math_tanh(x: int | float) -> float
def math_trunc(x: int | float) -> int
def math_ulp(x: int | float) -> float
```

## Deviations from native Python

We intend the Python §$x$ to be a conservative extension of native Python: Every correct Python §$x$ program should behave *exactly* the same using a Python §$x$ implementation, as it does using a native Python implementation. We assume, of course, that suitable libraries are used by the TypeScript implementation, to account for the predefined names of Python §$x$. This section lists some exceptions where we think a Python §$x$ implementation should be allowed to deviate from the native Python specification, for the sake of internal consistency and esthetics.

**Output Behavior Differences Between py-slang and Standard Python REPL:** Python provides an interactive mode, where users can type and immediately evaluate Python expressions line by line. This mode is entered when you simply type python in a terminal without specifying a file. This is commonly referred to as the Python REPL. In the standard Python REPL, any evaluated expression automatically has its result printed to the console, even if the user does not explicitly call `print()`. For example:

```
>>> 1 + 2
3
>>> "hello"
'hello'
```

In contrast, py-slang implements the behavior of **non-interactive** Python, the mode used when you pass a Python program to `python`. Only expressions explicitly passed to `print()` produce output. If the expression is evaluated without a print call, no output will appear, even though a value is computed.

For example, in py-slang:

```
print(1 + 2)     # Outputs: 3
```

But

```
1 + 2            # Outputs nothing
```

**Tail calls:** A call in tail position runs in constant space, so that a recursive function can run for any number of iterations: `f(10000)` with a tail-recursive `f` works, where native Python raises a `RecursionError` at a depth of about 1000.

**Conditions and logical operators:** The conditions of `if`, `elif` and `while`, and the operands of `not`, and the first operand of `and` and `or`, must be booleans, as the tables of operator types specify; native Python accepts any value (truthiness). `if 1:` and `0 or 'x'` are errors.

**Dynamic type checks:** The operators only accept the types of the tables of operator types. Native Python accepts more: `True + True`, `'ab' * 3`, `[1] < [2]` and `min([3, 1, 2])` are errors here.

**Numbers inside lists:** Numbers are compared by value, also when they are inside a list or pair, so that a `nan` in a list equals every `nan` in the other list, also one that was computed separately (native Python says False for two nans that were computed separately).

**Limits of some implementations:** These differences from native Python are due to the current state of the implementations, not to the language, and will be removed.

- A power with a float exponent that is not an integer can differ from Python's in the last digit, and a negative base with a fractional exponent gives `nan` instead of a complex number.
- The escape sequence `\U` of strings is not read.
- The implementations that run on the Pynter virtual machine (PVML, native Pynter) and the WebAssembly implementation evaluate the list, the index and then the value of `list[index] = value`; native Python evaluates the value first.
- The WebAssembly implementation has 64-bit integers, a lower limit for the depth of recursion, and does not support `global` declarations (a program with one is rejected).